

<i>ID</i>	<i>Description</i>	<i>Type</i>	<i>Priority</i>	<i>Description</i>	<i>Notes</i>	<i>Status</i>
PB2-01	Implement feature branch process	Technical capability	Must	Adopt a feature-branch workflow for Sprint 2 development to enable safe collaboration and controlled merges.	PR reviews + protected main branch	Done
PB2-02	Automated bike data collection (every 5 mins)	Technical capability	Must	Retrieve live station occupancy data from the JCDecaux API every 5 minutes.	Core data pipeline requirement	Done
PB2-03	Automated weather data collection (current/hourly/daily)	Technical capability	Must	Retrieve current, hourly, and daily weather data from the OpenWeather API on schedule.	Required for weather display + analytics	Done
PB2-04	Error handling for API requests	Technical capability	Must	Add robust error handling for bike and weather API calls (timeouts, non-200 responses, malformed JSON).	Resilience & reliability	Done
PB2-05	Transform JSON into structured data	Technical capability	Must	Parse raw JSON responses and transform into validated structured records ready for persistence.	Normalize fields and types	Done
PB2-06	Identify required data fields	Technical capability	Must	Identify required bike and weather fields needed for storage, API responses, and UI display.	Defines data contract	Done
PB2-07	Design ER diagram	Technical capability	Must	Design an ER diagram describing entities/relationships for bike stations, bike snapshots, and weather data.	Supports schema implementation	Done
PB2-08	Define database schema	Technical capability	Must	Define tables, columns, primary keys, foreign keys, and indexes for bike + weather data.	Schema spec	Done
PB2-09	Implement database schema in project	Technical capability	Must	Implement the designed schema in the project database (migrations/DDL).	Database ready for inserts	Done
PB2-10	Insert bike data into database	Technical capability	Must	Insert scraped bike station/snapshot data into the database with correct timestamps and station identifiers.	Persist bike history	Done
PB2-11	Insert weather data into database	Technical capability	Must	Insert scraped weather current/hourly/daily data into the database with correct timestamps.	Persist weather history	Done
PB2-12	Error handling and logging improvements	Technical capability	Must	Improve logging and error reporting across scraper, DB inserts, and API layer.	Operational visibility	Done
PB2-13	Test live API data retrieval	Technical capability	Must	Validate end-to-end retrieval from JCDecaux and OpenWeather APIs using real live calls.	Integration test	Done

PB2-14	Implement retention cleanup	Technical capability	Should	Implement data-retention cleanup to remove older records per defined policy to manage storage.	Keeps DB size manageable	Done
PB2-15	Create Flask app skeleton	Technical capability	Must	Create Flask project structure with configuration, routing, and basic health endpoint.	Foundation for API + UI	Done
PB2-16	Flask - live bike data endpoint (JSON)	Technical capability	Must	Expose an endpoint returning live JCDecaux bike data as JSON.	Supports frontend live view	Done
PB2-17	Flask - stored bike data endpoint (JSON)	Technical capability	Must	Expose an endpoint returning stored bike data from the database as JSON.	Supports historical view	Done
PB2-18	Flask - live weather data endpoint (JSON)	Technical capability	Must	Expose an endpoint returning live OpenWeather data as JSON.	Supports frontend live view	Done
PB2-19	Flask - stored weather data endpoint (JSON)	Technical capability	Must	Expose an endpoint returning stored weather data from the database as JSON.	Supports historical view	Done
PB2-20	Preliminary website creation	User story	Must	Create a basic website to host the dashboard and integrate with the Flask backend.	Initial UI scaffold	Done
PB2-21	Display live bike and weather data on webpage	User story	Must	Display live bike availability and live weather conditions on the webpage using Flask endpoints.	Core UX requirement	Done
PB2-22	Display bike stations on map	User story	Must	Show bike station locations on an interactive map with real-time availability.	Map visualization	Done
PB2-23	Test frontend data integration	Technical capability	Must	Test that frontend correctly consumes backend JSON endpoints and renders updates accurately.	UI/API integration test	Done
PB2-24	Deploy Flask application locally for testing	Technical capability	Should	Run and validate the full stack locally before cloud deployment.	Pre-deployment verification	Done
PB2-25	Configure environment variables (API keys)	Technical capability	Must	Configure environment variables for secrets (JCDecaux/OpenWeather keys) for local and EC2 environments.	Secure config	Done
PB2-26	Set up EC2 instance	Technical capability	Must	Provision and configure an AWS EC2 instance for hosting the application.	Cloud runtime	Done
PB2-27	Deploy application to EC2	Technical capability	Must	Deploy Flask application and dependencies to EC2 so it is accessible remotely.	Cloud deployment	Done
PB2-28	Deploy scheduled data collection on EC2	Technical capability	Must	Deploy and schedule the scrapers on EC2 (cron/systemd) to run continuously.	Automated collection in cloud	Done

PB2-29	Test live deployment	Technical capability	Must	Verify application, endpoints, and scheduled collectors work in the live EC2 environment.	Production smoke test	Done
--------	----------------------	----------------------	------	---	-----------------------	------